Diseño de Soluciones · Segundo corte

Sistema de gestión de barbería

API REST: endpoints y métodos HTTP

Grupo 8 · Daniel Felipe Vanegas y Camilo Andrés Paternina

El sistema está dividido en dos microservicios hechos con Spring Boot. Cada uno corre en su propio puerto, tiene sus propios controllers y guarda sus datos en su propio esquema de una base de datos PostgreSQL en Supabase: catalogo y citas. A continuación listamos los endpoints de cada controller.

Microservicio	Puerto	Responsabilidad
api-catalogo	8081	Servicios que ofrece la barbería y barberos que los atienden.
api-citas	8080	Clientes, citas y horario de atención. Para agendar una cita consulta a api-catalogo.

Microservicio api-catalogo (puerto 8081)

ServicioController

Microservicio	Método	Endpoint	Descripción
Catálogo	**POST**	`/api/servicios`	Crear servicio
Catálogo	**GET**	`/api/servicios`	Listar servicios (JSON o XML según el header Accept)
Catálogo	**GET**	`/api/servicios/{id}`	Consultar servicio (JSON o XML según el header Accept)
Catálogo	**PUT**	`/api/servicios/{id}`	Actualizar servicio
Catálogo	**DELETE**	`/api/servicios/{id}`	Eliminar servicio (se rechaza si tiene citas programadas: lo pregunta a api-citas)

BarberoController

Microservicio	Método	Endpoint	Descripción
Catálogo	**POST**	`/api/barberos`	Crear barbero
Catálogo	**GET**	`/api/barberos`	Listar barberos
Catálogo	**GET**	`/api/barberos/{id}`	Consultar barbero
Catálogo	**PUT**	`/api/barberos/{id}`	Actualizar barbero
Catálogo	**DELETE**	`/api/barberos/{id}`	Eliminar barbero (se rechaza si tiene citas programadas: lo pregunta a api-citas)

StatusController

Microservicio	Método	Endpoint	Descripción
Catálogo	**GET**	`/api/status`	Comprobar que el microservicio está en línea

Microservicio api-citas (puerto 8080)

ClienteController

Microservicio	Método	Endpoint	Descripción
Citas	**POST**	`/api/clientes`	Crear cliente
Citas	**GET**	`/api/clientes`	Listar clientes
Citas	**GET**	`/api/clientes/{id}`	Consultar cliente
Citas	**PUT**	`/api/clientes/{id}`	Actualizar cliente
Citas	**DELETE**	`/api/clientes/{id}`	Eliminar cliente (se rechaza si tiene citas programadas)

CitaController

Microservicio	Método	Endpoint	Descripción
Citas	**POST**	`/api/citas`	Agendar cita. Consulta el servicio y el barbero en api-catalogo y valida el horario de atención
Citas	**GET**	`/api/citas`	Listar citas. Filtros opcionales: clienteId, barberoId, servicioId, estado y fecha
Citas	**GET**	`/api/citas/{id}`	Consultar cita
Citas	**PUT**	`/api/citas/{id}`	Actualizar cita (reprogramar, cambiar servicio o barbero)
Citas	**PATCH**	`/api/citas/{id}/estado`	Cambiar solo el estado: completar o cancelar la cita
Citas	**DELETE**	`/api/citas/{id}`	Eliminar cita

HorarioController

Microservicio	Método	Endpoint	Descripción
Citas	**GET**	`/api/horario`	Consultar el horario de atención de los siete días de la semana
Citas	**PUT**	`/api/horario/{diaSemana}`	Cambiar la apertura y el cierre de un día (las dos en null: ese día no se atiende)

ConexionController

Microservicio	Método	Endpoint	Descripción
Citas	**GET**	`/api/conexion/catalogo`	Comprobar desde api-citas que api-catalogo responde

StatusController

Microservicio	Método	Endpoint	Descripción
Citas	**GET**	`/api/status`	Comprobar que el microservicio está en línea

Comunicación entre microservicios

Los dos microservicios se consultan con RestClient y la comunicación es síncrona: quien pregunta espera la respuesta antes de continuar. api-citas consulta a api-catalogo al agendar, y api-catalogo consulta a api-citas antes de eliminar un servicio o un barbero. Si el otro microservicio está apagado, se responde 503.

Quién consume	Método	Endpoint	Para qué
api-citas	**GET**	`/api/servicios/{id}`	Obtener el servicio al agendar o actualizar una cita
api-citas	**GET**	`/api/barberos/{id}`	Obtener el barbero al agendar o actualizar una cita
api-citas	**GET**	`/api/status`	Verificar la conexión entre los dos backends
api-catalogo	**GET**	`/api/citas?servicioId={id}&estado=PROGRAMADA`	Saber si un servicio tiene citas programadas antes de eliminarlo
api-catalogo	**GET**	`/api/citas?barberoId={id}&estado=PROGRAMADA`	Saber si un barbero tiene citas programadas antes de eliminarlo

Códigos de respuesta que usamos

Código	Significado	Cuándo lo devolvemos
200 OK	La operación salió bien	Consultas, actualizaciones, cambios de estado y eliminaciones
201 Created	Se creó el recurso	Al crear con POST
400 Bad Request	Los datos enviados no son válidos	Campos obligatorios vacíos, precio en cero, fecha pasada, cita fuera del horario de atención
404 Not Found	El recurso no existe	ID desconocido, o servicio o barbero que no está en el catálogo
409 Conflict	La operación choca con los datos actuales	Documento o nombre repetido, barbero o cliente ocupado a esa hora, cita ya cerrada o que aún no empieza al completarla, servicio o barbero con citas programadas al eliminarlo
503 Service Unavailable	El otro microservicio no respondió	api-catalogo apagado al agendar una cita, o api-citas apagado al eliminar un servicio o un barbero